October 13, 2026

Alzheimer's Disease Research



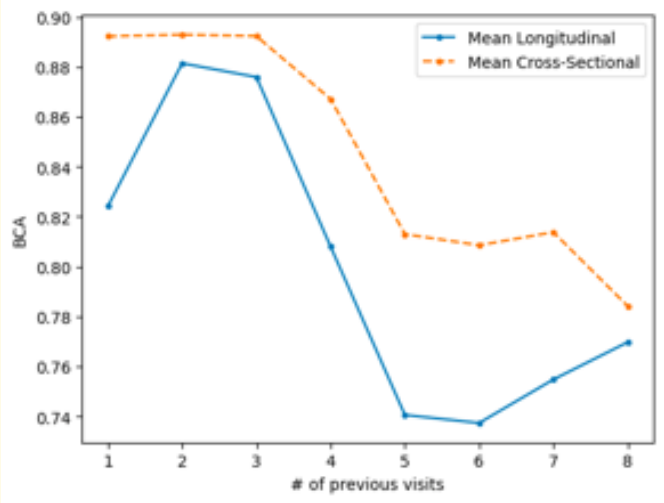
Mahdi Moghaddami

Projects Overview

Longitudinal vs. Cross-Sectional

- Longitudinal Models:
  - RNN
  - LSTM
  - GRU
  - MinimalRNN
- Cross-Sectional Models:
  - Random Forest
  - Support Vector Machine
  - Logistic Regression

- This study compares two categories of modeling: longitudinal and cross-sectional. We predict the clinical diagnosis of cognitive impairment in Alzheimer's disease (CN, MCI, or AD).



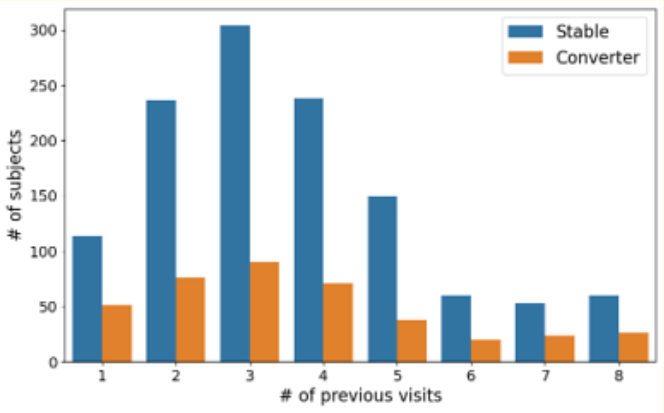


Figure 1: a) Mean BCA for top 3 models in each category, b) Data distribution based on the number of visits

Review Article

- Two major questions:
  1. How is longitudinal data used for AD?
  2. How should we write articles when there is an abundance of papers?
     - ASReview
     - Elicit
     - Covidence
     - DistillerSR
     - Nested-Knowledge

- 2020-2026 papers
- Four Sources:
  1. ACM Digital Library
  2. IEEEXplore
  3. Scopus
  4. PubMed
- We filter papers based on the existence of certain keywords in the title and abstract
- We find relevant papers.
- We need further ranking of paper. We add different metric for papers:
  - Journal papers:
    - Journal Impact Factor (JIF)
    - CiteScore
    - SCImage Journal Rank (SJR)
    - Source Normalized Impact per Paper (SNIP)
  - Conference papers:
    - Core Ranking (A*, A, B, C)
    - CCF Ranking (A, B, C)
  - Both:
    - Google Scholar h5-index

AD Progression Prediction using DTI

- The question:
  - Will a subject progress to a further stage of AD (CN $\longrightarrow$ MCI or MCI $\longrightarrow$ AD) in the next visit?
- Binary Stable/Converter prediction.

- 337 subjects
- The data:
  - DTI
  - Structured clinical and biomarker data (TADPOLE)
- Brett will talk more about this project!

AD Progression Prediction using fMRI

What is fMRI?

- Functional Magnetic Resonance Imaging is 4D and low-resolution brain scan that shows **which areas are active during specific tasks or at rest**.
- It's acquired by tracking blood flow changes. When a brain area is more active, it uses more oxygen, and fMRI can detect that via something called the **BOLD signal** (Blood Oxygen Level Dependent signal).
- Each voxel contains a BOLD signal that changes over time.
- It's basically **a number of 3D images**.

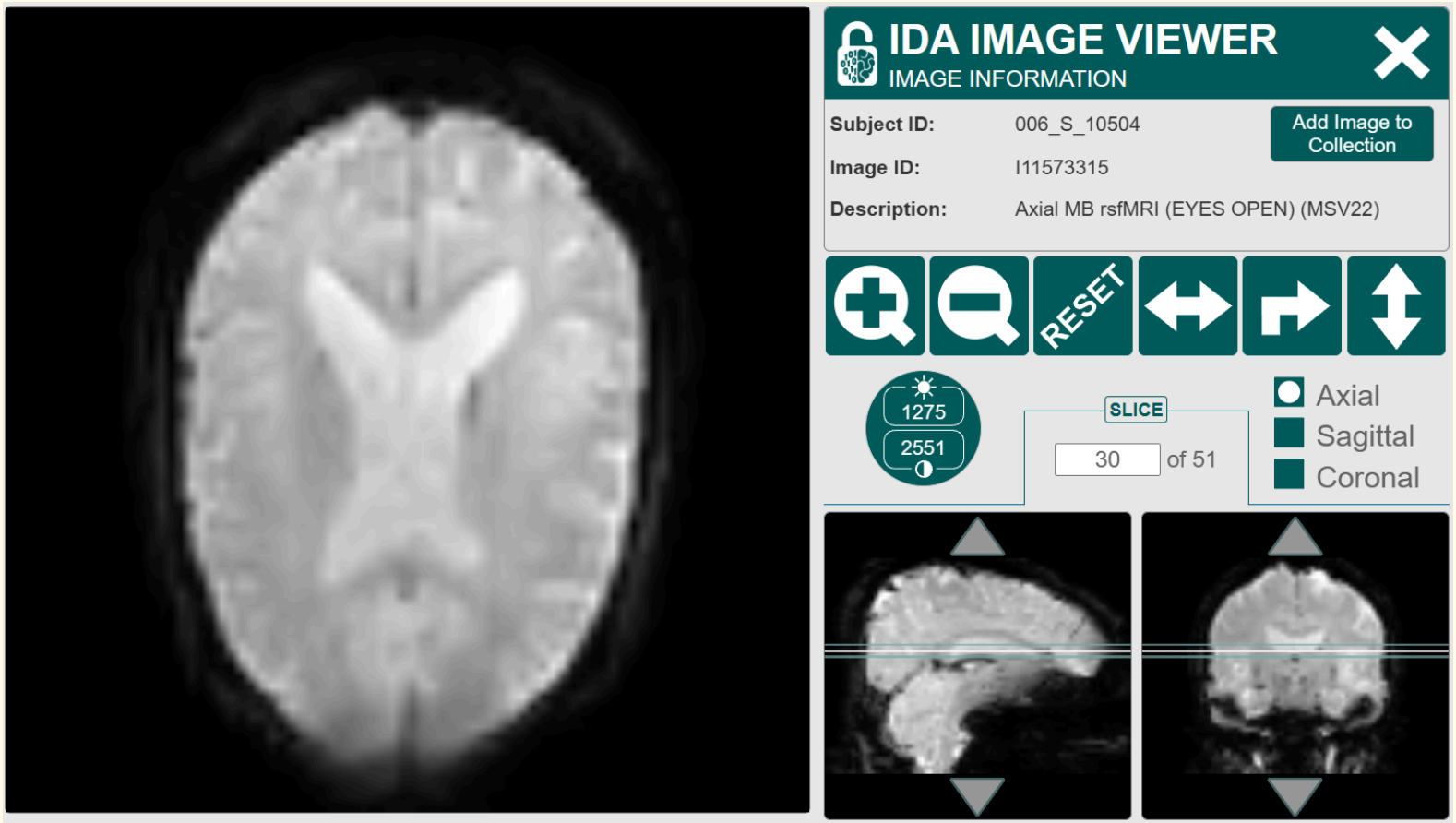


Figure 2: Example fMRI volume

Preprocessing Steps using fMRIPrep [Nature Methods 2019]

1. **Convert the data into a standard format:**
   - DICOM (`.dcm`) ⟶ NIfTI (`.nii`)
2. **Motion correction:**
   - During a several-minute fMRI scan, the subject inevitably moves a little.
   - fMRIPrep estimates how each volume moved relative to a reference volume.
   - It then realigns the volumes.
3. **Slice-timing correction:**
   - An fMRI volume isn't actually acquired all at once.
   - The scanner acquires different slices at slightly different times.
   - Slice-timing correction estimates what the signal would have been at the appropriate timing.
4. **Susceptibility distortion correction:**
   - Magnetic fields get warped near air-filled spaces in the skull (like the sinuses).
   - This step calculates that magnetic warping and "un-squishes" the image back to its true anatomical shape.
5. **Coregistration:**
   - fMRI images are low-resolution and blurry.
   - To know exactly where the brain activity is happening, fMRIPrep aligns the functional image with the subject's anatomical image (T1w MRI).
6. **Spatial normalization:**
   - Every person's brain is a slightly different size and shape.
   - fMRIPrep estimates a nonlinear transformation that maps the individual's anatomy to a standard brain template (usually called MNI space).

Generating Brain Functional Connectivity Network using NiLearn

1. **Parcellation (applying an atlas):**
   - Instead of analyzing 100,000 tiny individual voxels, Nilearn uses a brain "atlas" to group those voxels into larger, meaningful regions of interest (ROIs) (e.g., the hippocampus, the amygdala, the motor cortex).

$$R = \{R_1, R_2, ..., R_N\},$$

   set of ROIs defined by the atlas

2. **Extracting time series from each ROI (4D -> $N \times T$):**
   - Each voxel in an ROI has its own BOLD time series. We summarize these voxel signals, typically by taking the mean signal across voxels in the region.

$$R_1 = [x_1, x_2, ..., x_T],$$

   where T is the number of time points.

   - We do this for every region.

3. **Correlation matrix ($N \times T \rightarrow N \times N$):**
   - "How similarly does Region i's activity fluctuate with Region j's activity?"
   - Suppose:

$$R_i = [0.2, 0.4, 0.1, -0.2, ...]$$

$$R_j = [0.3, 0.5, 0.2, -0.1, ...]$$

   - We calculate the Pearson correlation:

$$r_{ij} = \text{corr}(R_i, R_j) = \frac{\sum (R_i - \mu_i)(R_j - \mu_j)}{\sqrt{\sum (R_i - \mu_i)^2} \sqrt{\sum (R_j - \mu_j)^2}}$$

   - $-1 \leq r_{ij} \leq 1$ for all $i, j \in \{1, ..., N\}$

FC Matrix

$$FC = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1N} \\ r_{21} & r_{22} & \cdots & r_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ r_{N1} & r_{N2} & \cdots & r_{NN} \end{bmatrix}$$

- An example functional connectivity (FC) matrix using the AAL atlas (number of regions: 116):

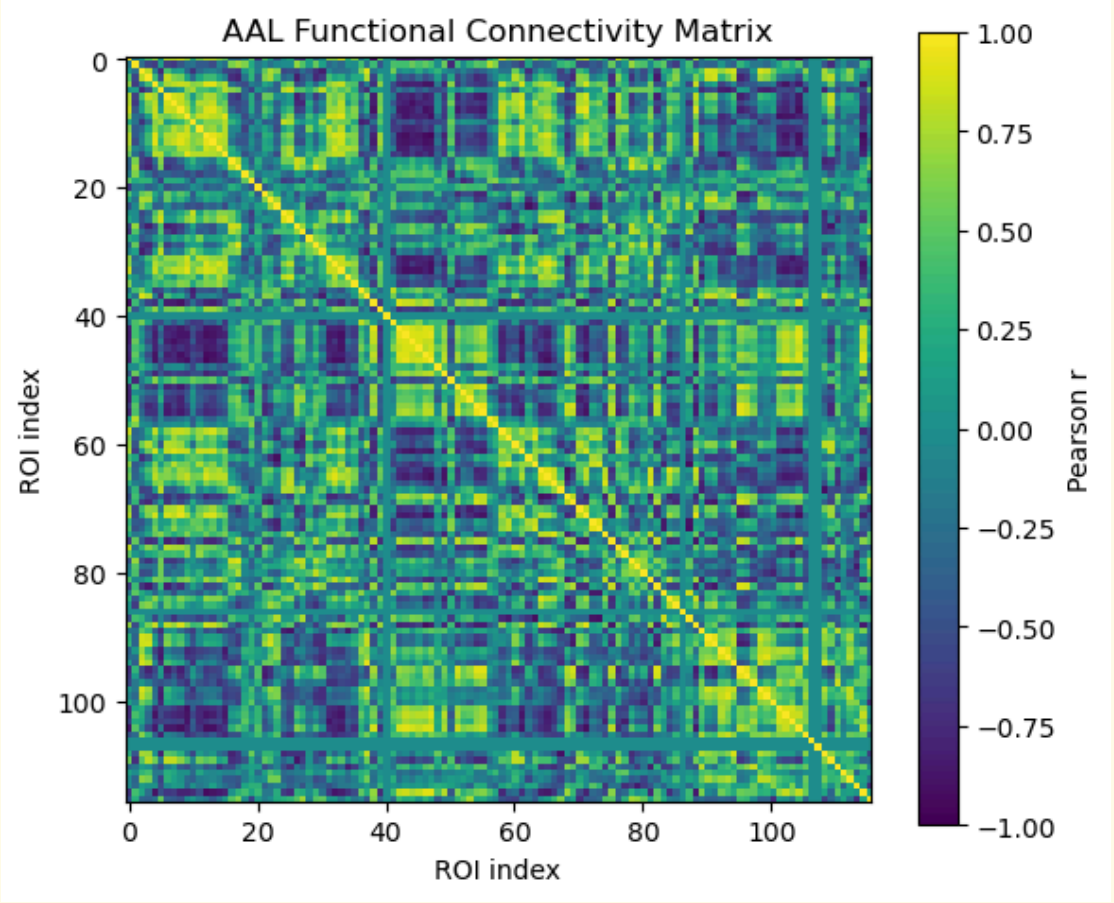


Figure 3: Example FC matrix

The Dataset

- **Stable sequence:** Last visit diagnosis same as previous
- **Converter sequence:** Last visit diagnosis different from previous (CN ⟶ MCI or MCI ⟶ AD)

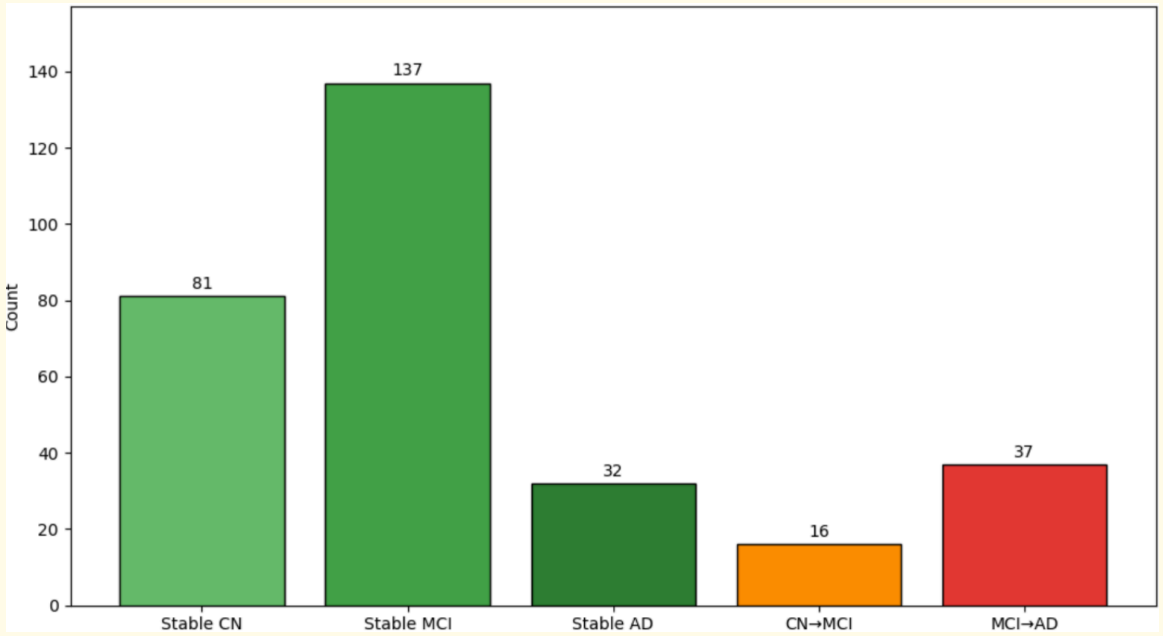


Figure 4: Stable/converter subject distribution

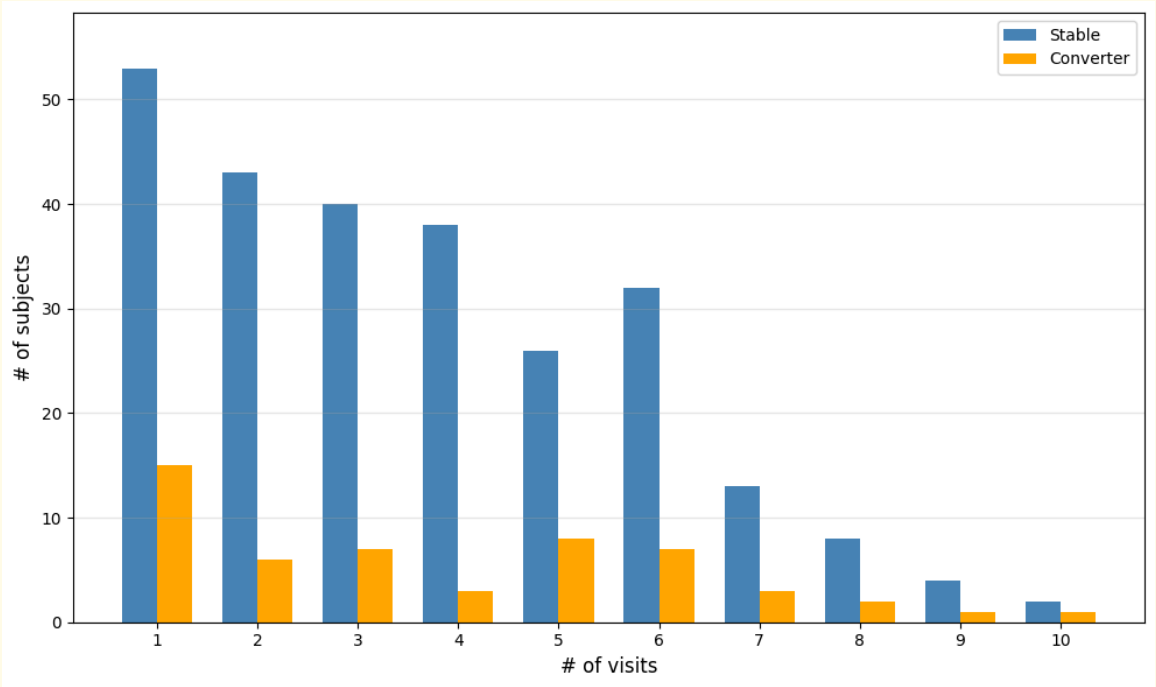


Figure 5: Another view of stable/converter subject distribution

The Model

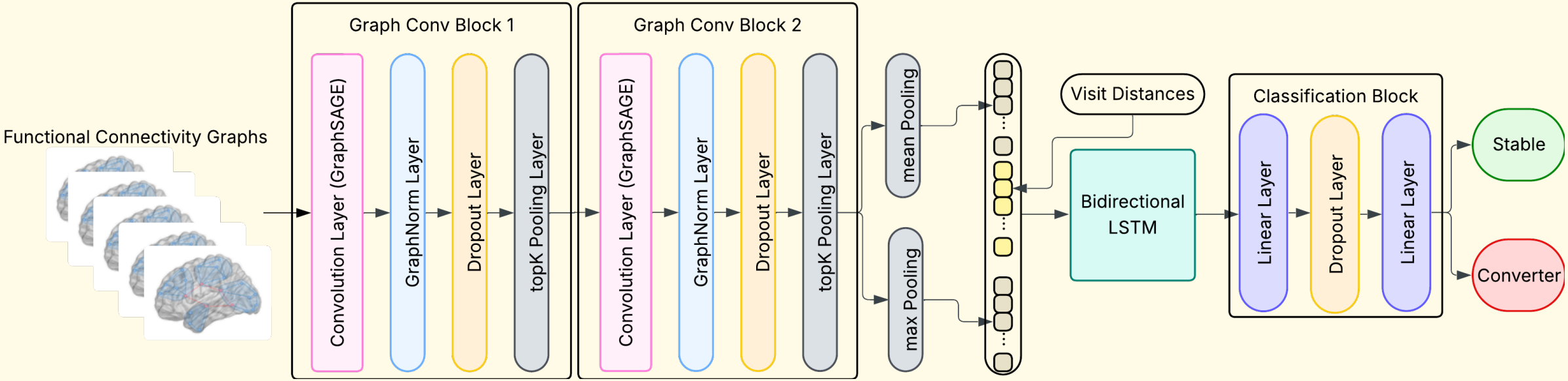
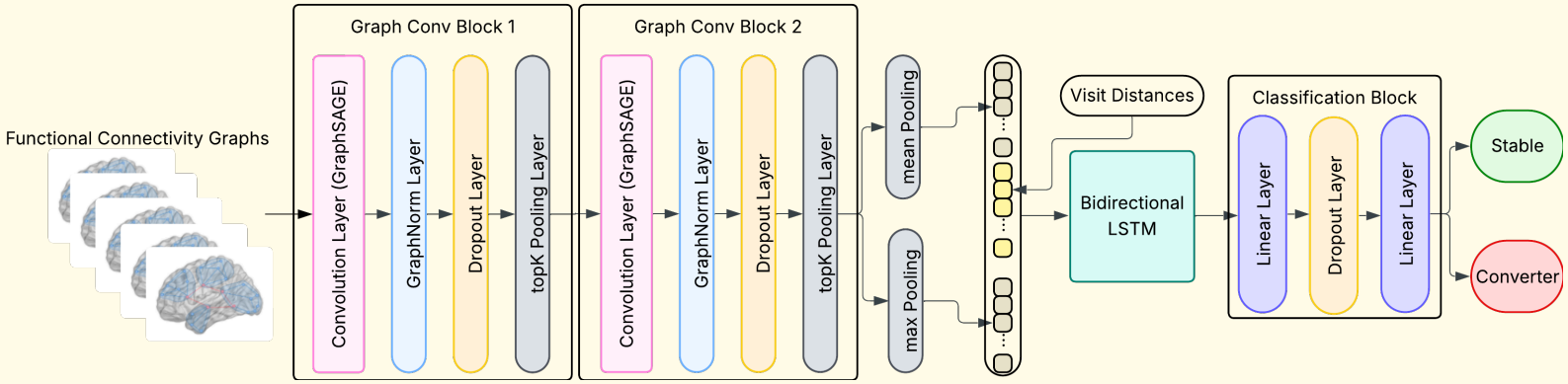


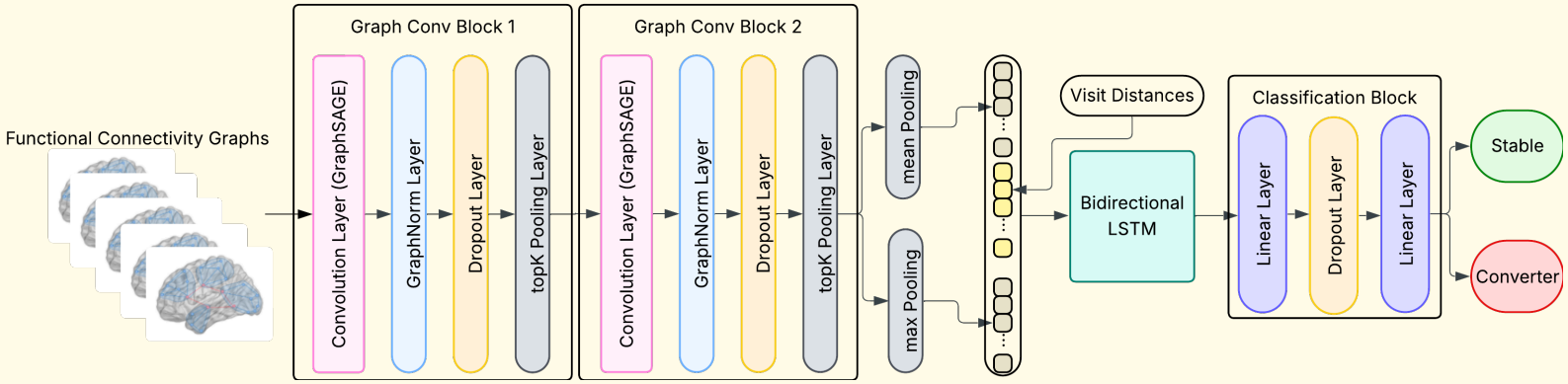
Figure 6: Model architecture

Input



- The original 2D functional connectivity matrix is $N \times N$, where N is the number of ROIs.
- For the GNN input, that matrix is converted into a graph G:
  - Nodes: V = ROIs
  - Edges: E = functional connections between ROIs
    - An edge exists only if the size of correlation between two nodes, $|r| \geq$ *threshold*. We set *threshold*=0.2.
  - Node features: X = connectivity profile of each ROI (its row of the FC matrix)
- The key difference is:
  - The 2D matrix is a dense table of pairwise correlations.
  - The GNN input is a graph object built from that matrix letting graph convolutions aggregate information over neighboring ROIs.
- The model input is multiple graphs for one subject, arranged as a visit sequence.
  - $[G_1, G_2, ..., G_m]$, where m is the number of visits.

GraphSAGE [NeurIPS 2017]



- For every node i:

$$x_i = [\text{corr}(i, R_1), \text{corr}(i, R_2), ..., \text{corr}(i, R_N)]$$

  - We aggregate neighbor features:

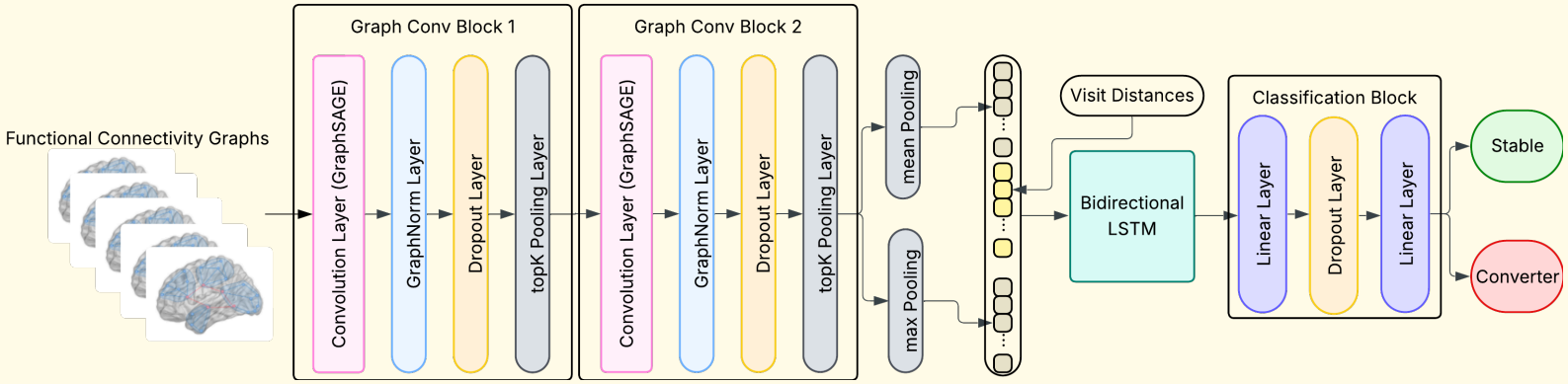
$$m_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} x_j$$

  - Combine node features with neighbor features:

$$h_i = \sigma\left(x_i W_{\text{self}}^T + m_i W_{\text{neigh}}^T + b\right)$$

- $h_i \in \mathbb{R}^{1 \times d}$: updated node embedding for node i
- σ: activation function
- $W_{\text{self}} \in \mathbb{R}^{d \times N}$: learned weight matrix for the node's own features
- $W_{\text{neigh}} \in \mathbb{R}^{d \times N}$: learned weight matrix for the aggregated neighbor features
- $b \in \mathbb{R}^{1 \times d}$: learned bias
- We do this for all N nodes.
  - Input: $X = [x_1, x_2, ..., x_N] \in \mathbb{R}^{N \times N}$
  - Output: $H = [h_1, h_2, ..., h_N] \in \mathbb{R}^{N \times d}$

GraphNorm [ICML 2021]



- Input: $H = [h_1, h_2, ..., h_N] \in \mathbb{R}^{N \times d}$
- GraphNorm normalizes the node embeddings within that one graph. In simple terms, it asks:
  - Are some node features unusually large or small compared with the rest of the graph?
- Then it rescales them, so the graph is easier for the next layers to process.
- Each node is normalized:

$$\hat{h}_i = \frac{h_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

  - ε is a small constant to prevent division by zero.

$$\mu = \frac{1}{N} \sum_{i=1}^{N} h_i \in \mathbb{R}^{1 \times d}$$

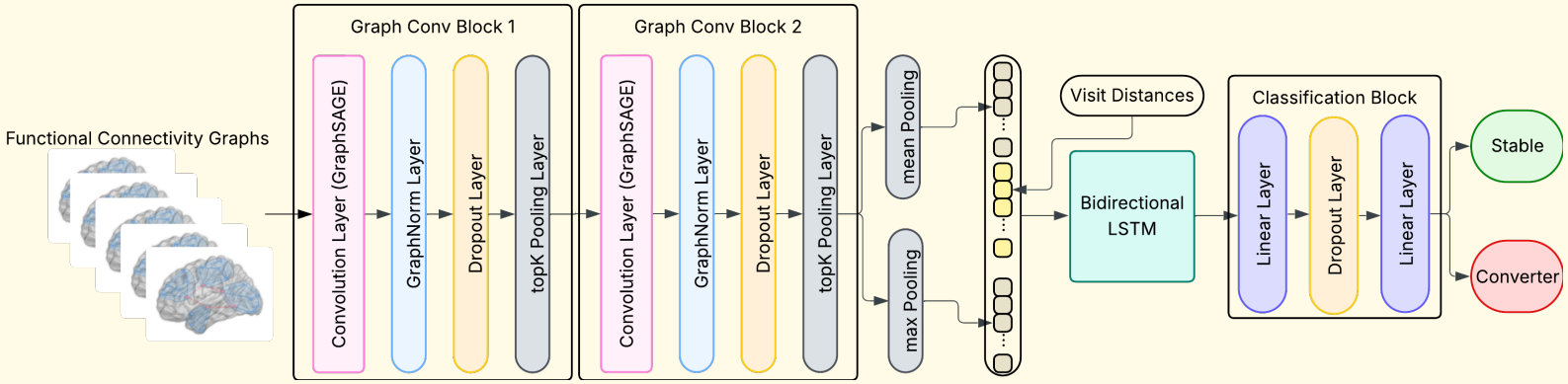
$$\sigma^2 = \frac{1}{N} \sum_{i=1}^{N} (h_i - \mu)^2 \in \mathbb{R}^{1 \times d}$$

- GraphNorm also learns a small rescaling and shifting for each feature:

$$h'_i = \gamma \odot \hat{h}_i + \beta$$

  - $\gamma, \beta \in \mathbb{R}^{1 \times d}$: learned parameters

Dropout [JMLR 2014]



- Input: $H' = [h'_1, h'_2, ..., h'_N] \in \mathbb{R}^{N \times d}$
- Dropout is regularization technique to prevent overfitting.
- During training, a random subset of neurons is deactivated.
- For each node i and feature k, we have a mask:

$$m_{i,k} \sim \text{Bernoulli}(1-p),$$

where p is the dropout probability.

$$h''_{i,k} = \frac{m_{i,k} \cdot h'_{i,k}}{1-p}$$

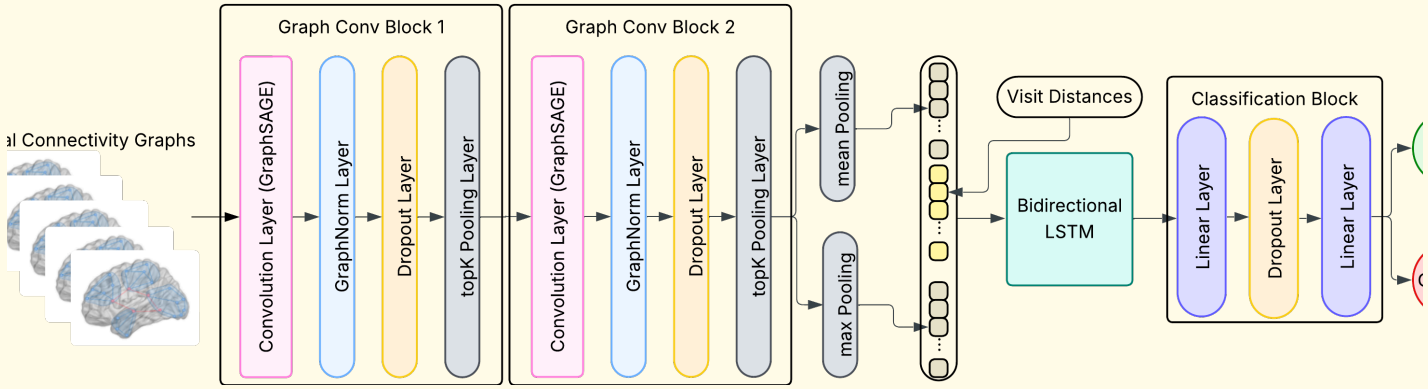
- Therefore:

$$H'' = \frac{H' \odot M}{1-p}$$

- Output:

$$H'' = [h''_1, h''_2, ..., h''_N] \in \mathbb{R}^{N \times d}$$

TopK Pooling [R2L@NeurIPS 2018]



- Input: $H'' = [h_1'', h_2'', ..., h_N''] \in \mathbb{R}^{N \times d}$
- TopK pooling keeps only the "most important" nodes and removes the rest.
- The model learns a score for each node. Top K highest-scoring nodes are kept; the rest are discarded.
- First, TopK pooling learns a projection vector $p \in \mathbb{R}^d$.
- Then, it computes a score for each node:

$$s_i = \frac{h_i''^T p}{\|p\|} \in \mathbb{R}$$

- Nodes are ranked based on their scores, and the top K nodes are selected.
- The model keeps only the top K nodes:

$$K = [rN],$$

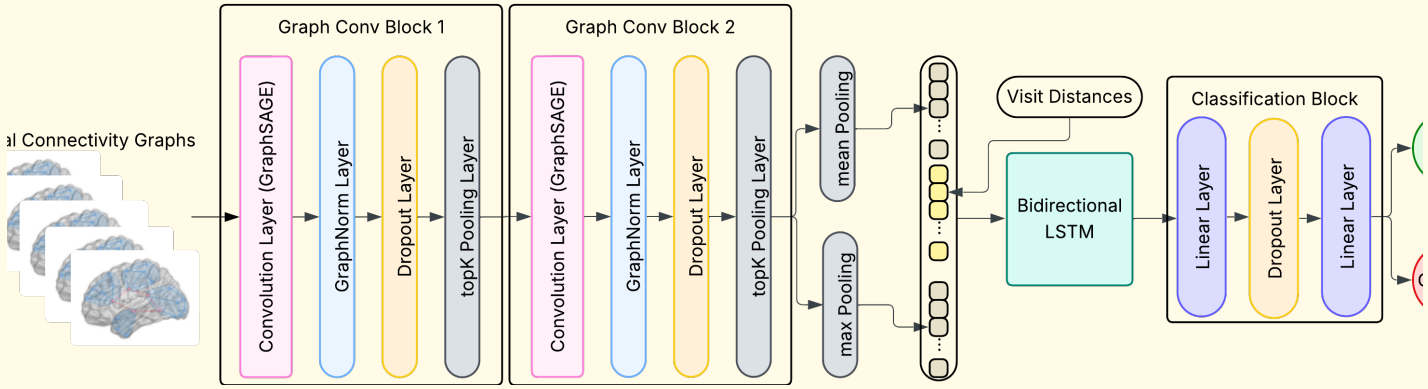
  where r is the pooling ratio (we use $r = 0.3$).
- Edges to discarded nodes are removed.
- For the kept nodes, TopK pooling gates their embeddings using the score to make the operation differentiable:

$$h_i^{\text{pool}} = h_i'' \cdot \tanh(s_i) \in \mathbb{R}^{1 \times d}$$

- Output:

$$H^{\text{pool}} = \left[h_1^{\text{pool}}, h_2^{\text{pool}}, ..., h_K^{\text{pool}}\right] \in \mathbb{R}^{K \times d}$$

Mean and Max Pooling



- Input:

$$H^{\text{pool}} = \left[h_1^{\text{pool}}, h_2^{\text{pool}}, ..., h_K^{\text{pool}}\right] \in \mathbb{R}^{K \times d}$$

- Mean pooling averages the values across nodes:

$$h^{\text{mean}} = \frac{1}{K} \sum_{i=1}^{K} h_i^{\text{pool}} \in \mathbb{R}^{1 \times d}$$

- Max pooling takes the maximum value across nodes, separately for each feature:

$$h^{\text{max}} = \max_{i=1}^{K} h_i^{\text{pool}} \in \mathbb{R}^{1 \times d}$$

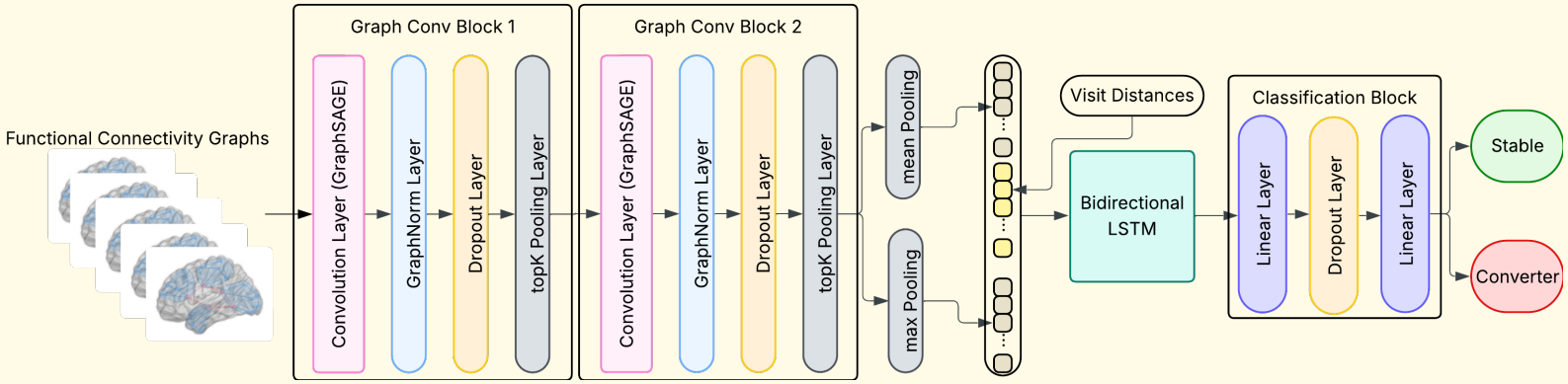
Example:

$$H^{\text{pool}} = \begin{bmatrix} 1 & 4 & 2 & 0 \\ 3 & 2 & 5 & 1 \\ 2 & 6 & 2 & 2 \end{bmatrix}$$

$$h^{\text{mean}} = [2, 4, 3, 1]$$

$$h^{\text{max}} = [3, 6, 5, 2]$$

Visit Distances



- Input:

$$\Delta \in \mathbb{R},$$

  showing the number of months until the next visit in the sequence.

- We apply logarithmic normalization:

$$\tau = \log\left(1 + \frac{\Delta}{12}\right)$$

- A small neural network turns that scalar into a 32-feature time embedding:

$$a = \text{Dropout}\big(\text{ReLU}(\tau W_1^T + b_1)\big) \in \mathbb{R}^{1\times 16}$$

$$e_{\text{time}} = aW_2^T + b_2 \in \mathbb{R}^{1\times 32}$$

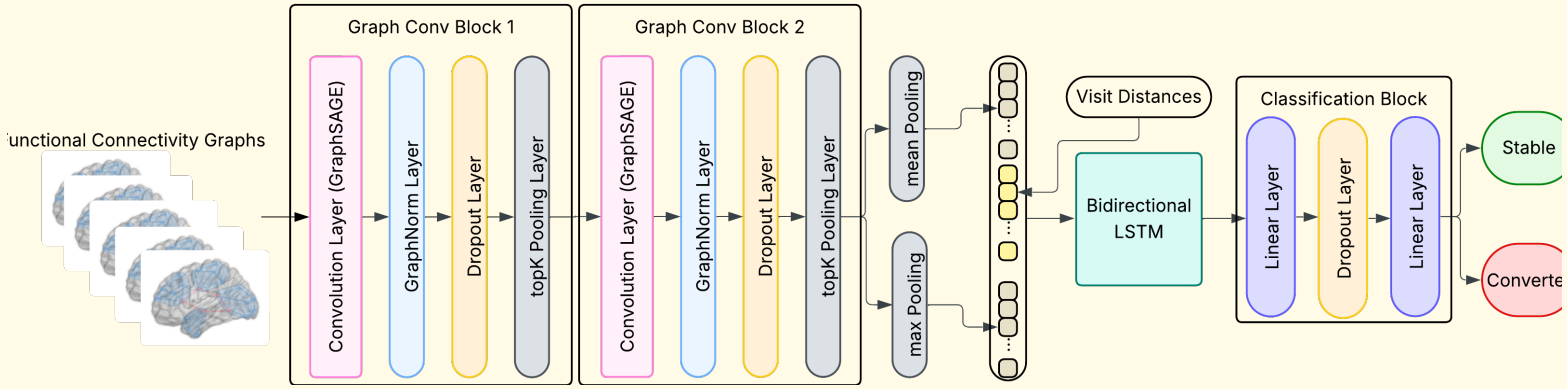
- We concatenate the time embedding with the graph representation:

$$u = [h^{\text{mean}}, e_{\text{time}}, h^{\text{max}}] \in \mathbb{R}^{1\times(2d+32)}$$

- A second learned layer mixes the graph and time information and projects it back to $2d$ features:

$$g = \text{Dropout}\big(\text{ReLU}(\tau W^T + b)\big) \in \mathbb{R}^{1\times 2d}$$

Bidirectional LSTM



- Input:

$$G = [g_1, g_2, ..., g_v] \in \mathbb{R}^{v \times 2d},$$

where v is the number of visits.

- The LSTM processes each embedding while maintaining a hidden state (h_t) and a memory state (c_t).
- the operations at visit t are:

$$i_t = \text{sigmoid}(g_t W_i^T + h_{t-1} U_i^T + b_i)$$

$$f_t = \text{sigmoid}\left(g_t W_f^T + h_{t-1} U_f^T + b_f\right)$$

$$o_t = \text{sigmoid}(g_t W_o^T + h_{t-1} U_o^T + b_o)$$

$$c_t' = \tanh(g_t W_c^T + h_{t-1} U_c^T + b_c)$$

- The memory and hidden state are updated:

$$c_t = f_t \odot c_{t-1} + i_t \odot c_t'$$

$$h_t = o_t \odot \tanh(c_t)$$

- Because the LSTM is bidirectional, one direction processes $(g_1 \rightarrow g_2 \rightarrow ... \rightarrow g_v)$, while the other processes $(g_v \rightarrow ... \rightarrow g_2 \rightarrow g_1)$. Each has its own learned weights.
- We concatenate the final hidden states from both directions to form the final visit sequence representation:

$$h = [h_v, h_1] \in \mathbb{R}^{1 \times 2d}$$

Pretraining

- Using 20% of the data

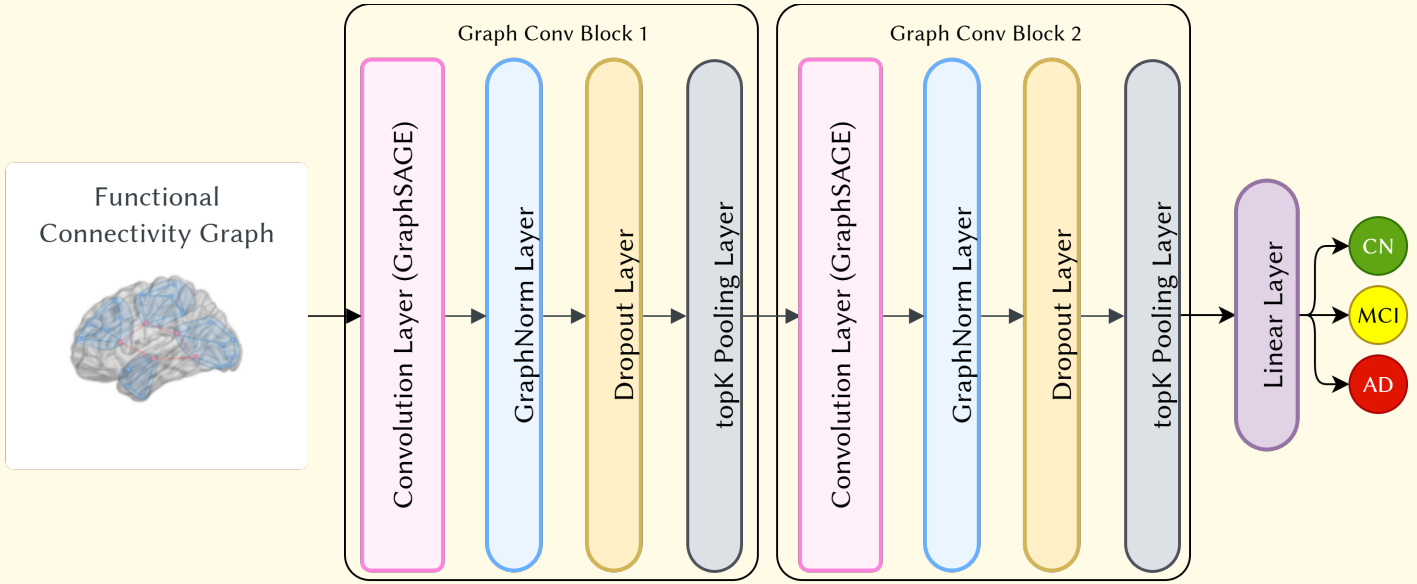


Figure 7: We pretrain the Graph convolution blocks.

Challenges

- Missing visits:
  - Some visits are missing fMRI but have data for other modalities (sMRI, cognitive scores, PET, etc.)
- Limited dataset:
  - We have only 303 subjects with fMRI data, and only 53 of them (17.5%) are converters.

Thank you!